Adaptive DIPF parameters (v13)

A learned gain-scheduling policy: the potential field still drives the car, a small model retunes four of its parameters every 0.1 s

1. The idea in one paragraph

Until v12, the DIPF parameters were constants for the whole run: one set (v9/v10), or one set chosen at the takeover (v11/v12). v13 makes four of them **functions of the current traffic**. Every control step a small learned policy reads nine dynamics features (closing speeds, required braking, gaps, lateral room, speed, density) and multiplies the base parameters up or down. The potential field still computes the acceleration exactly as before. The policy has 40 numbers and is trained **without labels**, directly on closed-loop segment success, with the evolution strategy CMA-ES. Control engineering calls this **gain scheduling**: controller gains looked up from measured operating conditions. Here the look-up table is learned.

2. Why adapt: evidence from the v12 library

The v12 run (in progress, 3 sets so far) found parameter sets that solve **different** scenarios. Their values differ by orders of magnitude: λ_{obs} is 0.033, 13, 322. Their outcomes are almost uncorrelated (0.05–0.27). On the validation scenarios the best single set succeeds 57% of the time; picking the right set per scenario would reach 70%. So no constant set fits all traffic, and the right values depend on the situation. v12 can only choose once per segment; v13 lets the parameters follow the situation as it changes.

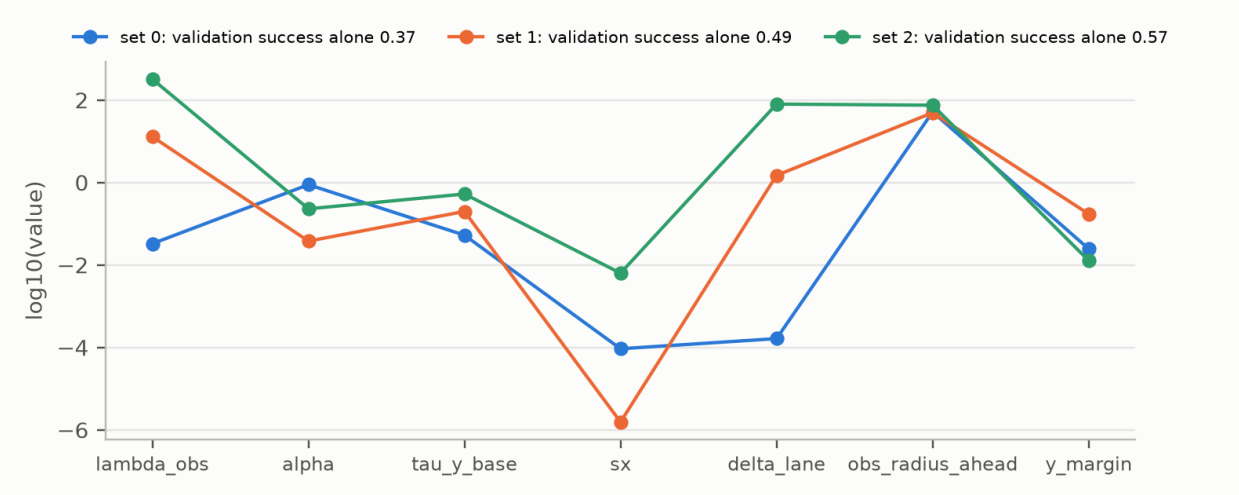


Figure 1. The parameter sets of the v12 library (log scale). Real data from v12/run1/library.json.

Two earlier lessons shape the design. (1) Per-scenario optimal parameters are not unique: many very different sets work equally well, so a regression on "best parameters" learned nothing ($R^2 \approx 0$). (2) A per-step model trained on states from fixed-parameter runs failed when it drove itself (v7), because its own driving led to states it had never seen, and its abrupt parameter switches hurt. v13 therefore uses no labels, trains on its own driving, and changes parameters smoothly.

3. What does not change: the potential decides every acceleration

At every step (0.1 s) the controller computes the potential `U = w_dest·U_dest + w_lane·U_lane + w_obs·Σ U_obs`, its gradient by central differences at the ego's position, and the acceleration `a = clamp(−∇U)`: +3.2 / −9 m/s² along the road, ±3 m/s² across, speed ≤ 34 m/s, sideways speed ≤ 3 m/s and ≤ v_x·tan 15°. There is no planner, no rule override and no safety filter. The policy only changes the parameters inside U; it never outputs an acceleration. A policy mistake can bend the field, but it cannot command an arbitrary manoeuvre.

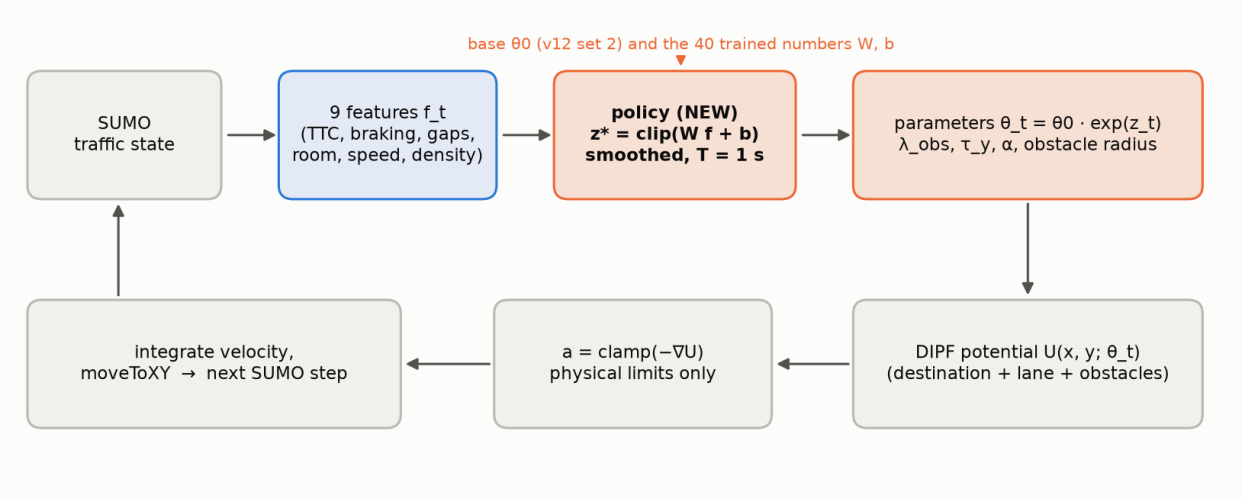


Figure 2. One control step in v13. The orange blocks are new; the rest is the unchanged DIPF controller.

4. The policy

Features. The 80-value traffic encoding is condensed into nine numbers, each scaled to [0, 1]. Each feature describes how dangerous or how crowded the situation is, not where vehicles are: the potential field already handles direction. All nine are unchanged when left and right are swapped, matching the symmetric controller.

feature	definition	meaning
ttc_front	$1 - e^{-1/TTC}$, vehicle in the ego's path ahead	closing in ahead (→1 when TTC < 1 s)
ttc_side_front	same, worst of front-left / front-right	closing in diagonally ahead
ttc_rear_side	same, worst of rear, rear-left/right, alongside	something closing from behind or the side
drac_front	$1 - e^{-DRAC/3}$, DRAC = closing² / (2·gap)	how hard the ego must brake
gap_front	$e^{-gap/20\ m}$ (0 if nothing ahead)	how close the vehicle ahead is
gap_side_front	same, closest of front-left / front-right	how close the diagonal vehicles are
tight_lateral	$e^{-room/1\ m}$, room = smallest of road-edge clearance and side gaps	room to swerve
speed	v_x / 34 m/s	ego speed
density	$1 - e^{-n/10}$, n = vehicles within ±50 m	crowding

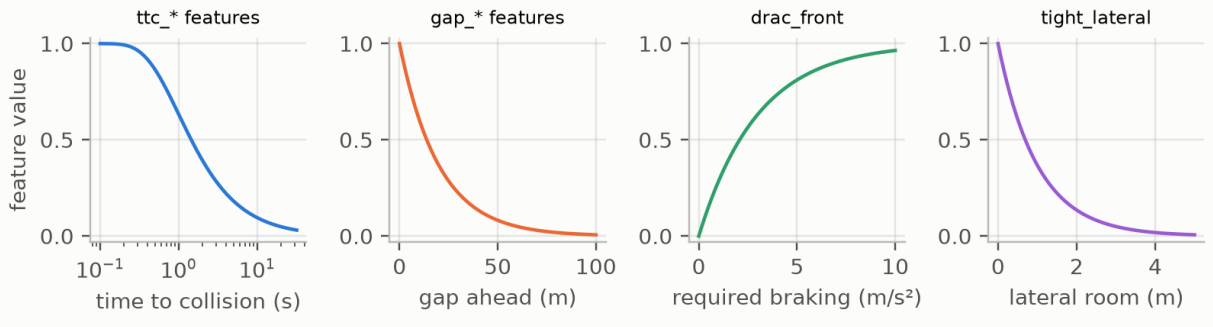


Figure 3. The transforms map unbounded quantities to [0, 1]: "no vehicle" and "far away" both give 0 (no jump), and the values saturate where they stop mattering.

Mapping. One exponent per adapted parameter, smoothed in time, then a multiplier on the base value:

```
z*_t = clip(W·f_t + b, −ln 10, +ln 10)      W: 4×9, b: 4 → 40 numbers
z_t  = z_(t−1) + (Δt/T)·(z*_t − z_(t−1))    T = 1 s, Δt = 0.1 s; first step z = z*
p_t  = clamp(p0·exp(z_t), limits)           p: λ_obs, τ_y, α, obstacle radius
```

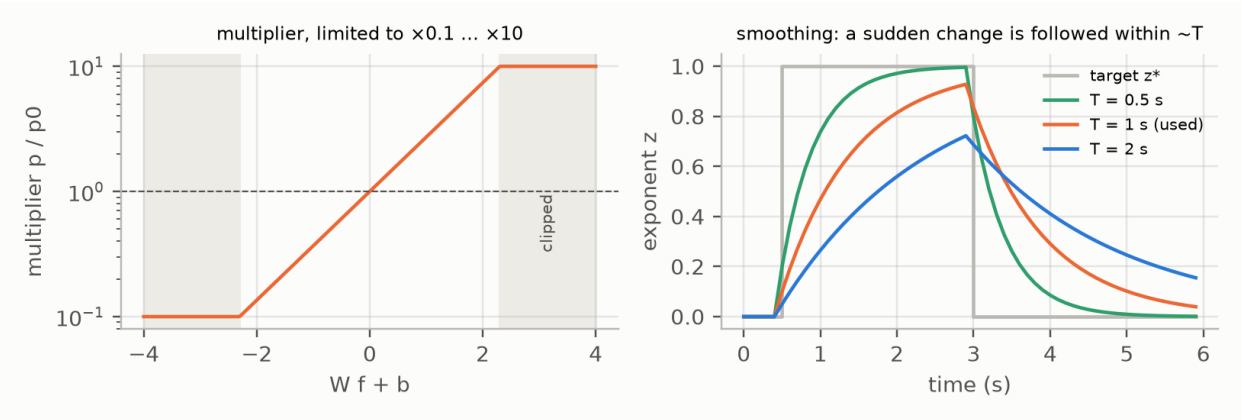


Figure 4. Left: the exponential gives multiplicative changes (natural for parameters spanning orders of magnitude, always positive); the clip keeps every parameter within ×0.1 ... ×10 of its base. Right: smoothing turns a sudden change of the target into a drift over about T.

adapted parameter	base p0 (v12 set 2)	absolute limits	role in the potential
λ_obs (lambda_obs)	322.1	0.01 ... 10000	strength of every obstacle's repulsion, U_obs = λ / k′
τ_y (tau_y_base)	0.5276	0.01 ... 20	lateral scale in k′: larger → vehicles to the side repel more
α (alpha)	0.2308	0.01 ... 100	how strongly τ changes with the relative velocity Δv (the dynamics-informed part)
obs_radius_ahead	75.24 m	30 ... 250 m	how far ahead obstacles are taken into account

The other tuned parameters (s_x, δ_{lane}, y_{margin}) stay at their base values; τ_x stays fixed at 1. A worked example: TTC = 1 s to the vehicle ahead gives ttc_front = $1 - e^{-1} = 0.63$. With a weight of 1.5 on (λ_{obs}, ttc_front) the exponent rises by 0.95, so λ_{obs} is multiplied by $e^{0.95} = 2.6$ while the danger lasts, and drifts back once the gap opens.

Safeguards built into the form. W = 0, b = 0 gives *exactly* the fixed controller. This was checked run by run: identical outcome and progress in every run. So training starts from the best fixed set and has to earn every change. Optional sign constraints keep the policy physically sensible: by default λ_{obs} may only grow (never shrink) as ttc_front, ttc_side_front or drac_front rise. The clip and the absolute limits bound every parameter, and the smoothing bounds how fast it moves.

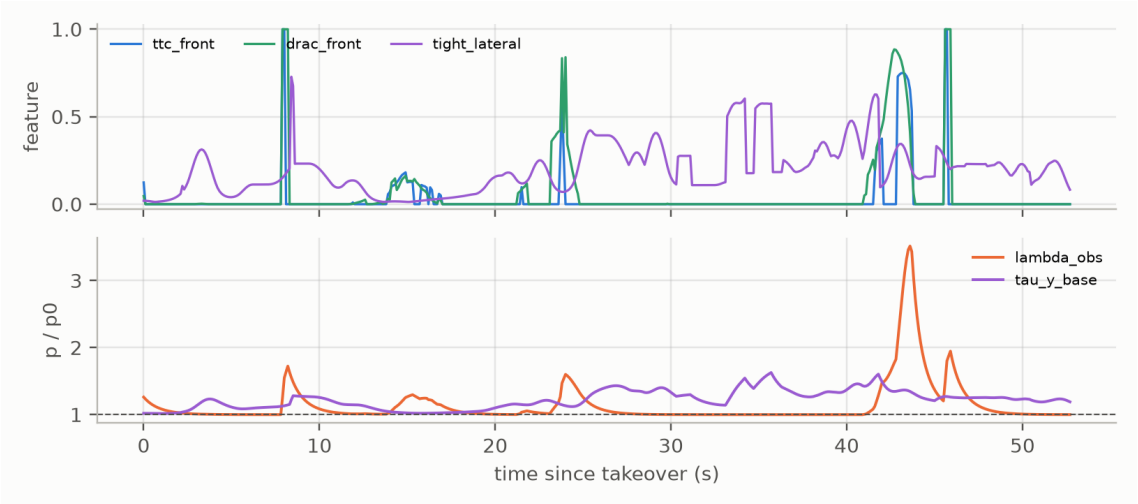


Figure 5. Mechanism demo, NOT a trained result: a real SUMO run of test scenario s05003 (success) with hand-set weights (λ_obs: +1.5 per ttc_front, +1.0 per drac_front; τ_y: +1.0 per tight_lateral). Repulsion rises when the ego closes in ahead and relaxes afterwards; the lateral scale follows the available room.

5. Training: closed-loop search instead of labels

There is no correct parameter value at time t to learn from: many values work, and a choice only shows its effect seconds later. So the policy is judged by what matters:

```
J(W, b) = success rate + 0.1 · mean progress
          over 100 train scenarios × 2 futures, each driven takeover → x = 400 m
```

J can only be measured by simulating, and it is noisy, so a gradient-free optimiser is used: **CMA-ES** (covariance matrix adaptation evolution strategy), the standard choice for tens of parameters. It keeps a Gaussian distribution over the 40 weights, starting at 0 (the base controller) with step size σ = 0.5. Each generation it samples 15 weight vectors, runs all of them on the same scenarios and futures (one parallel batch), moves its mean towards the better half, and stretches the distribution along directions that helped. It also learns correlations between weights, which matters when, say, raising λ_{obs} only helps together with a smaller obstacle radius. The implementation (common.py, NumPy) follows Hansen's tutorial and was checked on standard test functions.

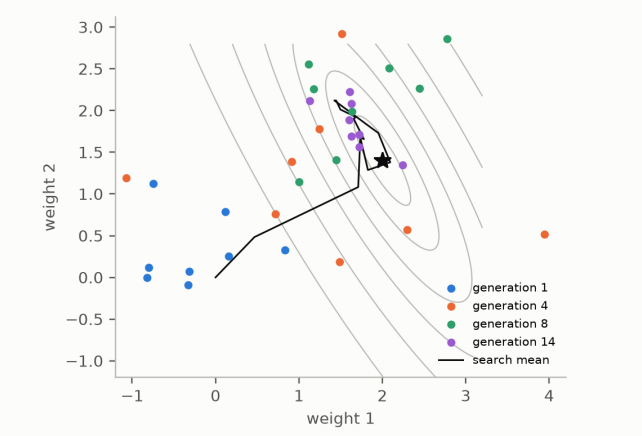


Figure 6. CMA-ES on a 2-D toy problem with correlated weights (the real v13 class). The sample cloud stretches along the valley and contracts around the optimum (star).

setting	value	note
dimensions	40	4 parameters × (9 weights + 1 bias)
population	15 per generation	4 + 3 ln 40
generations	30 (resumable, --finish to stop early)	≈ 450 candidates
runs per candidate	100 scenarios × 2 futures = 200	≈ 3000 runs per generation
validation	100 scenarios × 3 futures, every 4 generations	scores the search mean and the best training candidate
final policy	best validation J among all checkpoints	may be the base itself (W = 0) if nothing generalises

Choosing the final policy on validation rather than training matters. The best training candidate is partly lucky (it was picked from hundreds), so its training score overstates it. Validation needs enough scenarios too: in a 10-scenario smoke test, even the policy picked on validation did worse than the base on the test scenarios. That is why the real run uses 100 training and 100 validation scenarios.

6. A side finding: identical runs were not repeatable

While checking that W = 0 reproduces the fixed controller, identical runs (same saved state, same seed, same commands) sometimes gave different results. Repeating one segment 10 times gave up to **4 different outcomes**, including success versus collision. The cause is inside SUMO: it orders some internal work by memory address, and Linux randomises memory addresses on every launch (ASLR). After one step, two vehicles 300 m apart had swapped their random speed perturbations. Starting SUMO with address randomisation disabled (the equivalent of `setarch -R`, done in `_kill_with_parent`) made all 30 repeats identical.

10 repeats of …	default (ASLR on)	v13 (ASLR off)
s00007, future 2	2 different outcomes	1
s00003, future 1	4 (2 successes, 2 collisions)	1
s00012, future 2	2	1

Why it matters: CMA-ES (and the DE/Optuna searches in v11/v12) compare candidates on the same scenarios and futures so that differences come from the parameters, not from luck. Without the fix, every run carried extra randomness. v11/v12 results remain valid on average but are noisier than intended; the same one-line fix can be applied there.

7. How the result is tested

The final comparison uses 300 fresh test scenarios (seeds 5000–5299, never used in training) × 3 futures. Every policy meets exactly the same traffic, so results are compared run by run with McNemar's exact test (on runs where only one of the two succeeded). Three comparisons separate *adapting* from *re-tuning*:

policy	what it is	question it answers
base	fixed v12 set 2 (W = 0, b = 0)	reference
adaptive	the trained policy	adaptive vs base = total gain
bias part	the trained b with W = 0: a fixed re-tuning	adaptive vs bias part = gain from adapting during the run
bias-only (trained)	CMA-ES on b alone, same machinery (--bias-only)	adaptive vs bias-only = clean test: adaptation vs the best fixed re-tuning

If *adaptive* beats *base* but not *bias-only*, the gain came from better constant values, not from adaptation. Step 4 then shows what was learned: the weight map (each cell is a log-multiplier per unit feature, readable as physics, e.g. "double the repulsion when hard braking is needed"), parameter traces on test runs, and how far the parameters actually move.

8. Running it

step	command	time (7 jobs)
1 scenarios	`python step1_make_scenarios.py --n-jobs 7`	~2 min (or copy v12's scenarios)
2 train	`python step2_train_policy.py --n-jobs 7`	~8 h, resumable
control	`python step2_train_policy.py --bias-only --generations 15`	~2 h
3 test	`python step3_evaluate.py --n-jobs 7`	~20 min
4 figures	`python step4_analyze.py --n-jobs 7`	~2 min
watch	`python dipf_controller.py --episode scenarios/s05003 --policy policy.json --gui`	

Everything is in `v13/run1` (`./run_pipeline.sh 7` runs it all). Settings: `config.yaml` → `pipeline.adaptive` (base values, adapted parameters, limits, clip, smoothing, sign constraints) and `pipeline.cmaes` (search). Computing the features every step costs about 20% extra simulation time (≈ 2 s per segment run per worker).

9. What to expect, and limits

- **Noise.** With 300 test scenarios × 3 futures, differences below about 2–3 percentage points are hard to confirm. Use the McNemar p-values.
- **Unsolvable scenarios stay unsolvable.** If no parameter values avoid a crash (v12 set aside 34 such scenarios in its pool), adapting the parameters cannot help either. Only something outside the potential could, such as a safety filter or a learned acceleration correction.
- **Linear and small by design.** 40 weights, smooth and bounded, so the result can be read and checked. If it helps, the next steps are a tiny MLP (e.g. 9 → 8 → 4) trained the same way, more features, or reinforcement learning with the parameter changes as actions.
- **Compute.** About 8 h per training run. Fewer adapted parameters, fewer generations or fewer scenarios in early generations reduce it.
- **Base matters.** The policy rescales v12 set 2. A different base (config.yaml → pipeline.adaptive.base_params) gives a different starting point; training must be restarted.